파이썬 프로그래밍

한국외국어대학교 Social Science & AI융합학부

이 동 현

Copyright©2026 By Donghyun Lee. All right reserved.

목차

- ✓소개
- ✓강의 소개
  - ✓강의 목표
  - ✓강의 계획

강의소개

- ✓수업 목표
  - ✓파이썬 프로그래밍
    - ✓1. 파이썬 프로그래밍 기초
      - ✓자료형, 연산자, 조건문, 반복문, 함수
    - ✓2. 파이썬 프로그래밍 기본
      - ✓클래스, 모듈, 패키지, 예외 처리, 입출력

강의소개

- ✓강의 계획
  - ✓1. 파이썬 프로그래밍
    - ✓1.1. 파이썬 소개 및 설치, 기초
    - ✓1.2. 자료형 (변수)
    - ✓1.3. 연산자
    - ✓1.4. 조건문 및 반복문
    - ✓1.5. 함수
    - ✓1.6. 입출력

강의소개

- ✓1.7. 파이썬 스타일 코드
- ✓1.8. 클래스
- ✓1.9. 모듈 및 패키지
- ✓1.10. 예외 처리

Reference

- ✓Byte of python

  http://swaroopch.com/notes/python

- ✓KOCW 파이썬 프로그래밍
- ✓KOCW 빅데이터 분석
- ✓KOCW 프로그래밍 기초

파이썬 소개 및 설치, 기초

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓파이썬 소개

✓프로그래밍 기초

✓파이썬 설치

파이썬 소개

- ✓파이썬(Python):
  - ✓귀도 반 로섬(Guido van Rossum)이 1991년 개발
  - ✓인터프리터(Interpreter) 언어

- ✓주 활용분야:
  - ✓개발
  - ✓분석

파이썬 소개

- ✓장점:
  - ✓간결함
  - ✓오픈 소프트웨어
  - ✓다양한 패키지(머신러닝 패키지 등)
  - ✓개발과 분석이 모두 가능

- ✓단점:
  - ✓일부 느린 연산 속도
  - ✓앱 개발 한계

파이썬 소개

- ✓개발 환경:
  - ✓프로그램:
    - ✓파이썬
  - ✓IDE(Integrated Development Environment): 통합개발환경
    - ✓Jupyter notebook
    - ✓Pycharm 등

파이썬 설치

- ✓파이썬 설치
  - ✓아나콘다 (Anaconda)
    - ✓파이썬, NumPy 등 주요 오픈 라이브러리를 한번에 설치하는 범용적으로 사용 중인 파이썬 배포 버전
    - ✓Anaconda3 4.4.0 다운 (4.4 이후 버전도 가능)
      - ✓파이썬3 버전 기준으로 진행
      - ✓https://repo.continuum.io/archive/
      - ✓Anaconda3-4.4.0-Windows-x86_64.exe 다운

아나콘다 설치

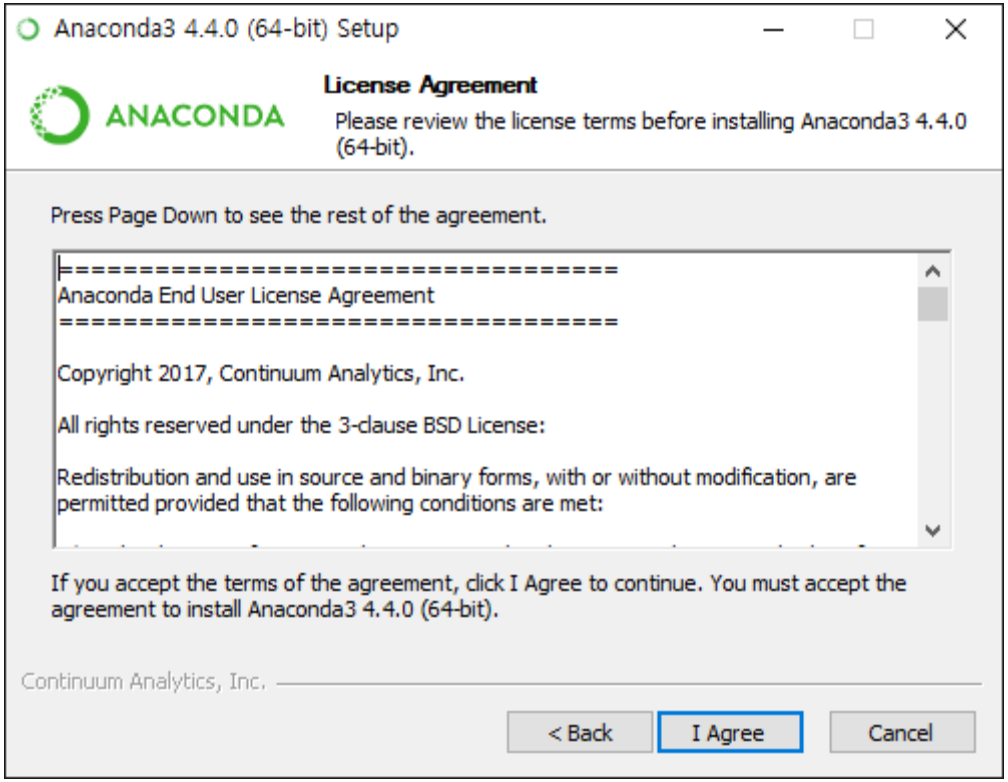
Anaconda3-5.0.0-Windows-x86_64.exe	510.0M	2017-09-26 14:14:53	fee3fad608d0006afa5c7bca4de3d02b
Anaconda3-5.0.0-Linux-ppc64le.sh	296.3M	2017-09-25 14:39:31	8fe5b29ca5be3ff11411621f79babfc2
Anaconda2-4.4.0.1-Linux-ppc64le.sh	271.4M	2017-07-26 16:10:02	ce166de6f116acd08cd313f9c55c04d6
Anaconda3-4.4.0.1-Linux-ppc64le.sh	285.6M	2017-07-26 16:08:42	fe7c87abd9fd70dc0cb4f83cc22d336f
Anaconda2-4.4.0-Linux-x86.sh	415.0M	2017-05-26 18:23:30	b0f8f5ade832b0238357c2f973338b17
Anaconda2-4.4.0-Linux-x86_64.sh	485.2M	2017-05-26 18:22:48	d72add23bc937ccdfc7de4f47deff843
Anaconda2-4.4.0-MacOSX-x86_64.pkg	438.0M	2017-05-26 18:36:08	d2d5d213764a0c849eb1d53acba0d418
Anaconda2-4.4.0-MacOSX-x86_64.sh	375.4M	2017-05-26 18:35:52	a57e5c631a7d0c63552519f05ab243a4
Anaconda2-4.4.0-Windows-x86.exe	354.4M	2017-05-26 17:52:15	51f14d30b08b82cd5e44bbb6b0d63349
Anaconda2-4.4.0-Windows-x86_64.exe	430.7M	2017-05-26 17:53:22	0f60aa52ef3a5d6170aeb6f7e3651f91
Anaconda3-4.4.0-Linux-x86.sh	428.7M	2017-05-26 18:23:45	8556e85f81206c08ee2a30b67d1bb707
Anaconda3-4.4.0-Linux-x86_64.sh	499.0M	2017-05-26 18:23:04	50f19b935dae7361978a04d9c7c355cd
Anaconda3-4.4.0-MacOSX-x86_64.pkg	442.5M	2017-05-26 18:36:17	c6cd9c30b94c2ba2a5449e6f234d15f5
Anaconda3-4.4.0-MacOSX-x86_64.sh	380.4M	2017-05-26 18:35:59	3958ac6cb84731e560dd833256aa5b15
Anaconda3-4.4.0-Windows-x86.exe	362.2M	2017-05-26 17:54:21	c7a66350b79354773dabbbef6f58a3af
Anaconda3-4.4.0-Windows-x86_64.exe	437.6M	2017-05-26 17:55:34	aa200a1c059a551e0ba9a5314a9554a5
Anaconda2-4.4.0-Linux-ppc64le.sh	276.6M	2017-05-17 15:45:20	511fdc6f6c29b1c3a702f3792182faf0
Anaconda3-4.4.0-Linux-ppc64le.sh	290.7M	2017-05-17 15:45:50	8c663e2056c7c57ac0075774b1f790be
Anaconda2-4.3.1-Linux-x86.sh	387.7M	2017-03-06 16:12:31	aae1a3192abee1f0abba6c0e1b292cec

Anaconda3-4.4.0-Windows-x86_64.exe 다운
(64비트일 경우임, 32비트의 경우 x86)

아나콘다 설치

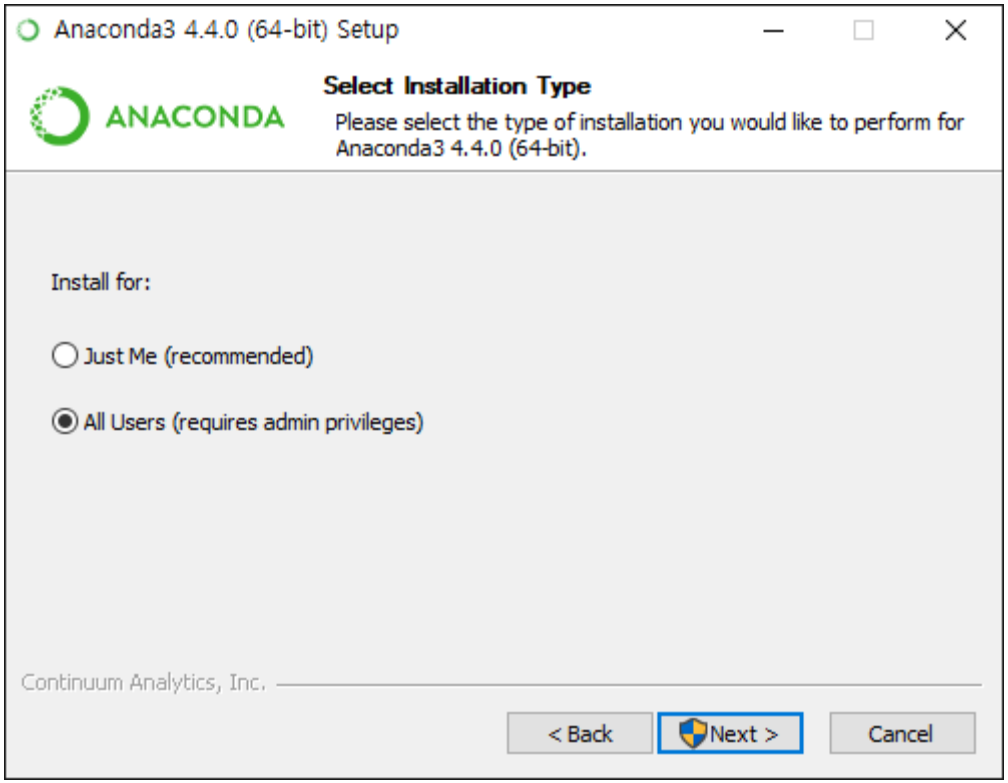
Anaconda3 4.4.0 설치

아나콘다 설치



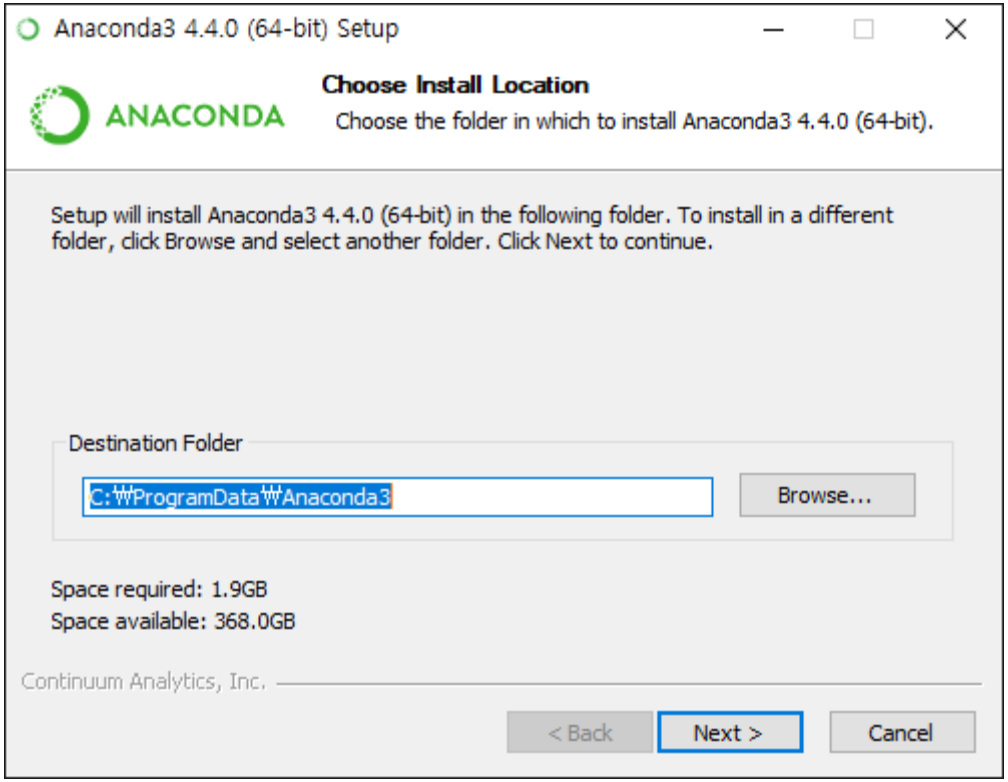
Anaconda3 4.4.0 설치

아나콘다 설치



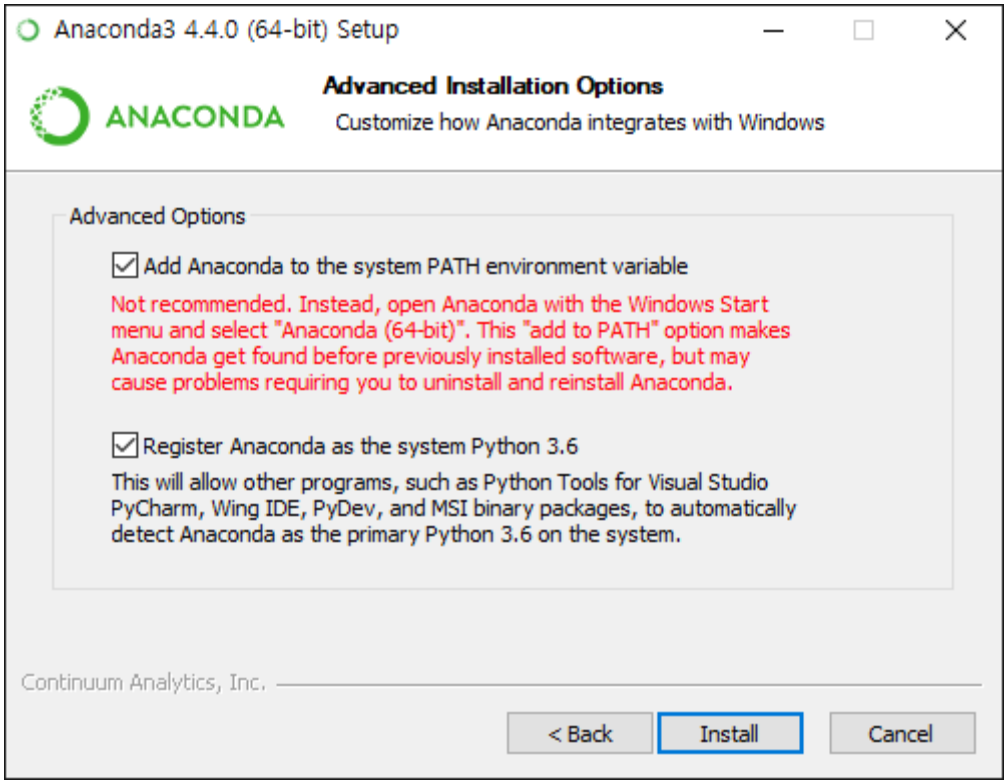
Anaconda3 4.4.0 설치

아나콘다 설치



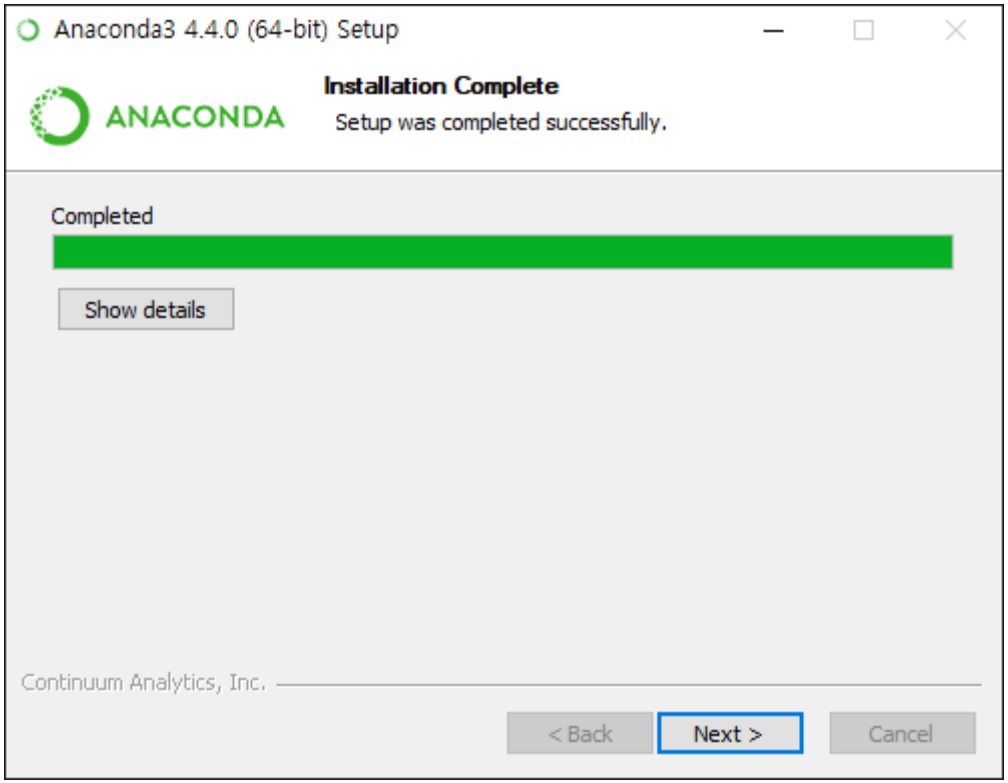
Anaconda3 4.4.0 설치

아나콘다 설치



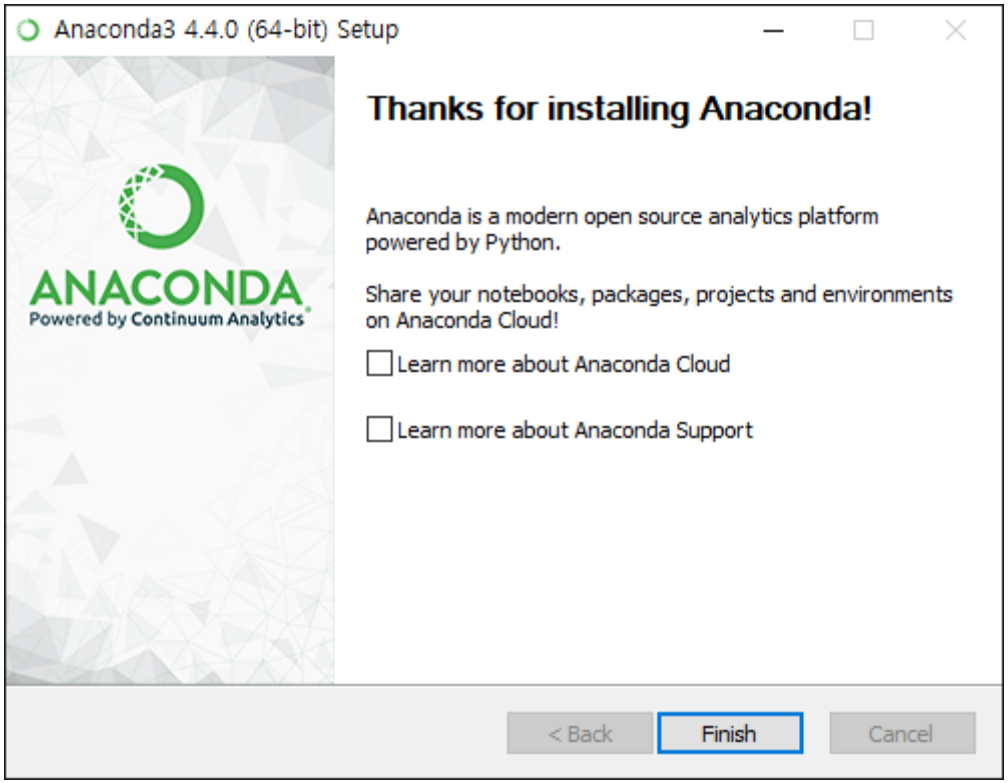
Anaconda3 4.4.0 설치

아나콘다 설치



Anaconda3 4.4.0 설치

아나콘다 설치



아나콘다 설치 완료

프로그래밍 기초 및 Jupyter notebook 사용법

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓프로그래밍 기초
- ✓Jupyter notebook 기초

프로그래밍 기초

- ✓프로그래밍(Programing):
  - ✓컴퓨터가 수행하는 명령어를 작성하는 것

- ✓프로그래밍의 장점:
  - ✓반복 가능

출처: KOCW 빅데이터 분석 -이동현

프로그래밍 기초

- ✓프로그래밍 유의사항:
  - ✓프로그램 명세 작성
    - ✓프로그램 목적, 기능, 논리 설명 및 정의
  - ✓주석
    - ✓실행 시 프로그램에 영향을 주지 않음
    - ✓본인, 협업
  - ✓들여쓰기(indent)
    - ✓레벨
    - ✓파이썬 필수
    - ✓스페이스 바(space bar) 4번 또는 Tab

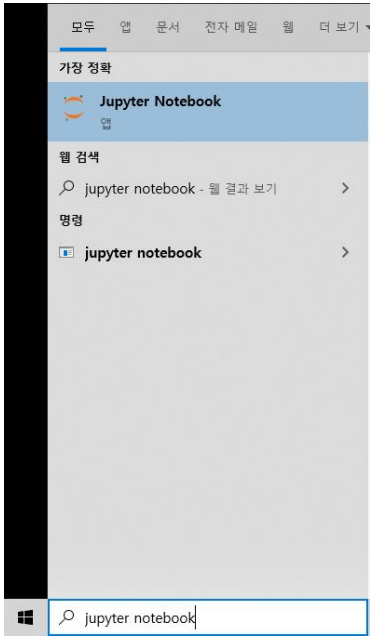
출처: KOCW 빅데이터 분석 -이동현

Jupyter notebook 기초

✓Jupyter notebook 실행

✓하단 검색에서 jupyter notebook 검색 실행 또는

시작 -> anaconda3 -> jupyter notebook 실행

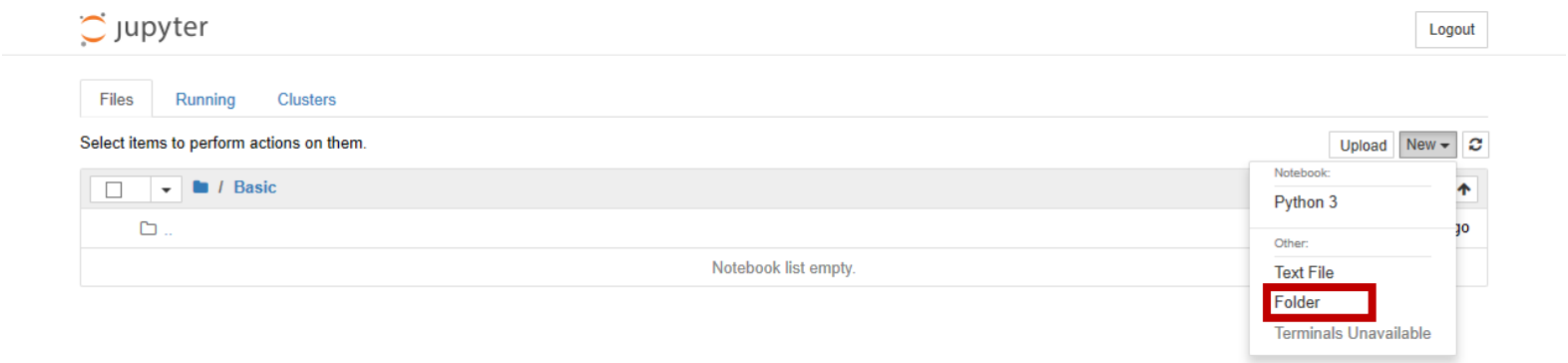


Jupyter notebook 실행

Jupyter notebook 기초

✓Jupyter notebook 작업 폴더 지정

✓New -> Folder: 작업 폴더 생성



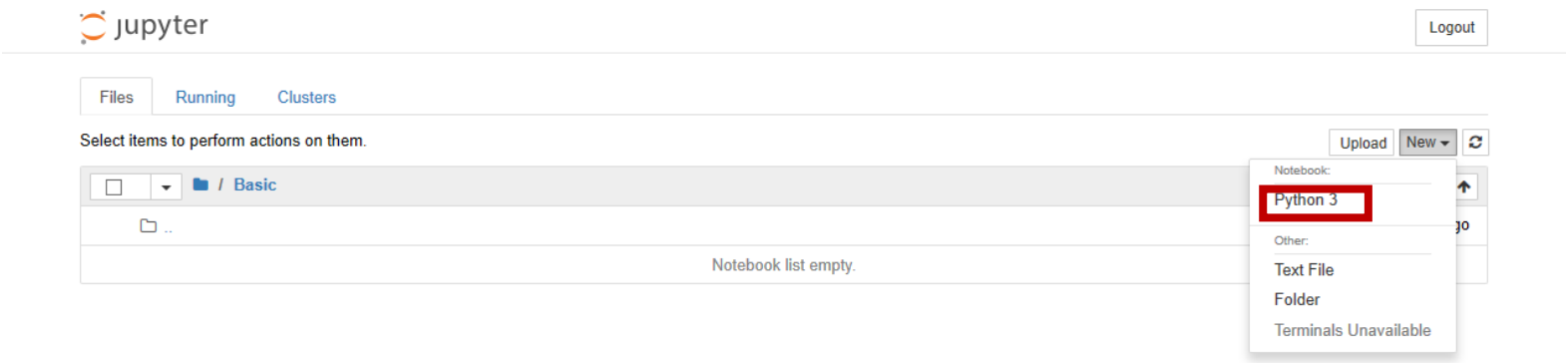
Jupyter notebook 화면

Jupyter notebook 기초

✓파이썬 파일 생성

✓New -> Python3

✓ipynb 확장자 파일 생성됨

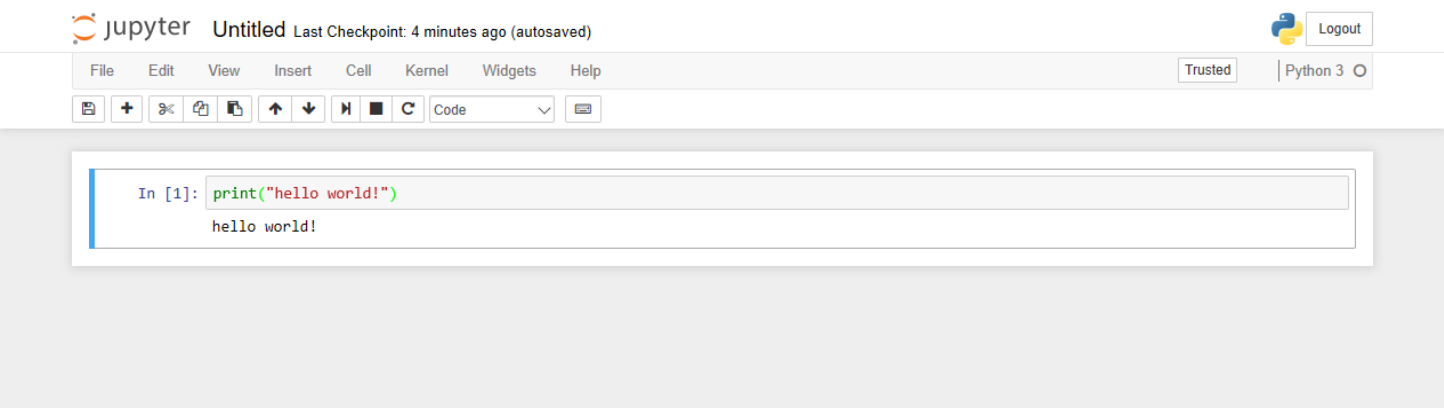


Jupyter notebook 화면

Jupyter notebook 기초

✓파이썬 실행

✓print('hello world!') 명령어 작성 후 Ctrl + Enter



Jupyter notebook 화면

Jupyter notebook 기초

- ✓ Command mode
  - ✓ Esc 또는 셀 바깥으로 마우스를 클릭
  - ✓ 파란색으로 표시

```
In [2]: print("hello world!")
        hello world!
```

- ✓ Edit mode
  - ✓ Enter 또는 마우스로 해당 셀을 클릭
  - ✓ 초록색으로 표시

```
In [2]: print("hello world!")
        hello world!
```

Jupyter notebook 기초

- ✓ Command mode
  - ✓ a 위로 셀 추가
  - ✓ b 아래로 셀 추가
  - ✓ dd 선택 셀 삭제
  - ✓ x 선택 셀 잘라내기
  - ✓ c 선택 셀 복사
  - ✓ v 선택 셀 붙여넣기
  - ✓ Shift + m 선택 셀과 아래 셀 병합
  - ✓ Enter Edit 모드로

Jupyter notebook 기초

- ✓ Edit mode
  - ✓ Ctrl + Enter 선택 셀 실행
  - ✓ Shift + Enter 셀 실행 후 다음 셀 이동
  - ✓ Shift + Ctrl + - 커서 위치 기준 셀 분할
  - ✓ Ctrl + / 커서 위치 줄 주석 처리
  - ✓ Tab 들여쓰기
  - ✓ Ctrl + s 파일 저장
  - ✓ Esc Command mode로

파이썬 자료형

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓자료형
- ✓선언과 할당
- ✓변수명
- ✓파이썬 자료형
- ✓Jupyter notebook 사용법 실습

자료형

- ✓자료형 (변수형)
  - ✓변수의 종류 또는 유형(type)
  - ✓변수: 데이터를 저장하는 공간

선언과 할당

- ✓**선언**
  - ✓필요한 자료형의 변수를 가져오는 것
  - ✓파이썬은 할당을 하면 적절한 자료형의 선언이 함께 이루어지므로 선언을 특별히 하지 않음
- ✓**할당 (대입)**
  - ✓변수에 데이터 혹은 특정한 값을 저장하는 것
  - ✓파이썬은 할당 시 = 을 사용
  - ✓처음 값을 할당을 하는 것을 **초기화**라고 함
  - ✓a, b = 1, 2 등으로 여러 변수 한번에 대입 가능함
  - ✓E.g.) Java의 경우 int a; a = 3; 파이썬의 경우 a = 3

변수명

- ✓**변수명**
  - ✓문자, 숫자, 밑줄(_)로 구성, 대소문자 구분함
  - ✓불가한 변수명
    - ✓예약어
      - ✓False, None, True, and, as, assert, break, class, continue, def, del, elif, else, except, finally, for, from, global, if, import, in, is, lambda, nonlocal, not, or, pass, raise, return, try, while, with, yield
    - ✓한글, 특수기호 / 첫 글자 숫자

변수명

- ✓**변수명**
  - ✓일관된 스타일의 변수명 활용 시 가독성 높일 수 있음
  - ✓일반 변수는 소문자, 상수는 대문자 추천
  - ✓l(소문자 엘), O(대문자 오), I(대문자 아이)는 구분이 힘들기 때문에 피할 것
  - ✓PEP 8(Python Enhancement Proposal) 참고

자료형

- ✓주요 내장 함수 (import 없이 사용 가능한 함수)
  - ✓print(변수)
    - ✓변수를 출력함
  - ✓type(변수)
    - ✓변수의 자료형을 알려줌
  - ✓len(변수)
    - ✓변수의 길이를 알려줌

파이썬 자료형

- ✓ 파이썬 자료형
  - ✓ 단일 자료형
    - ✓ 정수형(int)
    - ✓ 실수형(float)
    - ✓ 복소수형(complex)
    - ✓ 논리형(bool)
  - ✓ 군집 자료형
    - ✓ 문자열(str)
    - ✓ 리스트(list)
    - ✓ 튜플(tuple)
    - ✓ 집합(set)
    - ✓ 딕셔너리(dict)

Jupyter notebook 사용법 실습

✓Jupyter notebook 사용법 실습

파이썬 자료형 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓단일 자료형

복습

- ✓자료형(변수형)
  - ✓변수의 종류 또는 유형(type)
  - ✓변수: 데이터를 저장하는 공간

복습

- ✓ 파이썬 자료형
  - ✓ 단일 자료형
    - ✓ 정수형(int)
    - ✓ 실수형(float)
    - ✓ 복소수형(complex)
    - ✓ 논리형(bool)
  - ✓ 군집 자료형
    - ✓ 문자열(str)
    - ✓ 리스트(list)
    - ✓ 튜플(tuple)
    - ✓ 집합(set)
    - ✓ 딕셔너리(dict)

단일 자료형

- ✓ **단일 자료형**
  - ✓ **정수형(int)**
    - ✓ 정수
    - ✓ 산술연산 가능

단일 자료형

- ✓ **단일 자료형**
  - ✓ **실수형(float)**
    - ✓ 실수
    - ✓ 산술연산 가능
    - ✓ 소수점 이하 표현가능
    - ✓ 실수 2.0 표현에서 2. 등으로 0 생략가능

단일 자료형

- ✓ **단일 자료형**
  - ✓ **복소수형(complex)**
    - ✓ 복소수
    - ✓ 복소수 연산 가능

단일 자료형

- ✓ **단일 자료형**
  - ✓ **논리형(bool)**
    - ✓ 참, 거짓을 True, False 로 나타냄

파이썬 자료형 3

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓군집 자료형

복습

- ✓자료형(변수형)
  - ✓변수의 종류 또는 유형(type)
  - ✓변수: 데이터를 저장하는 공간

복습

- ✓ 파이썬 자료형
  - ✓ 단일 자료형
    - ✓ 정수형(int)
    - ✓ 실수형(float)
    - ✓ 복소수형(complex)
    - ✓ 논리형(bool)
  - ✓ 군집 자료형
    - ✓ 문자열(str)
    - ✓ 리스트(list)
    - ✓ 튜플(tuple)
    - ✓ 집합(set)
    - ✓ 딕셔너리(dict)

군집 자료형

- ✓ **군집 자료형**
  - ✓ **문자열(str)**
    - ✓ 문자열
    - ✓ 군집 형태로 처리하여 특정 위치 문자 인덱싱(indexing), 슬라이싱(slicing) 으로 추출 가능
    - ✓ 파이썬의 인덱싱은 0부터 시작하며 변수명[위치] 형태로 사용
    - ✓ 슬라이싱(slicing)은 변수명[시작위치:(끝위치+1)] 형태로 사용

군집 자료형

- ✓ 변수명[:b+1]는 해당 변수의 **시작부터** b까지 슬라이싱
- ✓ 변수명[a:]는 a부터 해당 변수 **끝까지** 슬라이싱
- ✓ 인덱싱 및 슬라이싱 시 음수는 끝을 기준으로 한 뒤로의 위치를 의미함
- ✓ 변수명[시작위치:(끝위치+1):stride] 형태로 사용 가능. 단, 복잡해 보일 수 있음

군집 자료형

- ✓ +, * 등의 문자열 더하기, 곱하기 가능함
- ✓ 따옴표 ‘, 큰따옴표 “”, ‘’ ‘’ 또는 “”” “”” 로 표현
- ✓ 문자열 포맷팅
  - ✓ 변화하는 변수와 결합하여 활용하기 유용
  - ✓ ‘hello world%d’ %1
  - ✓ %s: 문자열, %d: 정수형, %f: 실수형
  - ✓ 괄호를 활용하여 2개 이상 변수 표현 가능
    - ✓ ‘hello%d world%d’ %(1,5)
- ✓ 주요 메소드(methods)
  - ✓ help(변수명) 으로 확인 가능

군집 자료형

- ✓ **군집 자료형**
  - ✓ **리스트(list)**
    - ✓ 군집 자료형
    - ✓ 대괄호 [ ]로 사용
    - ✓ 인덱싱(indexing), 슬라이싱(slicing) 자주 활용
    - ✓ 인덱싱은 변수명[위치] 형태로, 슬라이싱(slicing)은 변수명[시작위치:(끝위치+1)] 형태로 사용 가능
    - ✓ 리스트 안의 리스트 가능

군집 자료형

- ✓ 변수명[:(**b+1**)] 는 해당 변수의 **시작부터 b까지** 슬라이싱
- ✓ 변수명[a:]는 **a부터 해당 변수 끝까지** 슬라이싱
- ✓ 변수명[시작위치:(끝위치+1):stride] 형태로 사용 가능.

군집 자료형

- ✓ 주요 메소드
  - ✓ append(a)
    - ✓ 데이터 a를 추가
  - ✓ insert(a, b)
    - ✓ a위치에 데이터 b를 넣음
  - ✓ pop()
    - ✓ 가장 끝 위치의 데이터를 빼냄

군집 자료형

- ✓ **군집 자료형**
  - ✓ **튜플(tuple)**
    - ✓ 수정 불가한 군집 변수

```
TypeError: 'tuple' object does not support item assignment
```

    - ✓ 소괄호 ( ) 로 사용
    - ✓ 인덱싱(indexing), 슬라이싱(slicing) 가능

군집 자료형

- ✓ **군집 자료형**
  - ✓ **집합(set)**
    - ✓ 집합을 표현할 때 활용
    - ✓ { } 또는 set([ ] 또는 “문자”)으로 표현
    - ✓ 순서가 없으며 요소로 관리
    - ✓ 합집합, 교집합, 차집합 등 집합연산 활용가능

군집 자료형

- ✓ **군집 자료형**
  - ✓ **딕셔너리(dict)**
    - ✓ 키(key): 값(value) 형태의 자료에서 활용
    - ✓ {key1: value1, key2:value2} 형태로 사용
    - ✓ 키는 유일한 값이어야 함
    - ✓ 주요 메소드
      - ✓ keys: key 종류를 알 수 있음
      - ✓ values: value 를 알 수 있음
      - ✓ Items: (key, value) 묶음을 받을 수 있음

파이썬 자료형 4

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓자료형 변환

✓군집 자료형 실습

자료형 변환

- ✓ 자료형 변환
  - ✓ 현재 자료형에서 다른 자료형으로 변환
  - ✓ 변수를 다른 자료형으로 변환 완료하려면 변환 후 할당 해야 함. e.g.) a = int(a)
  - ✓ 사용법
    - ✓ 변환할 자료형(변수)
    - ✓ e.g.) int(변수)

자료형 변환

- ✓ 정수형(int)
  - ✓ int(변수)
- ✓ 실수형(float)
  - ✓ float(변수)
- ✓ 복소수형(complex)
  - ✓ complex(변수)
- ✓ 논리형(bool)
  - ✓ bool(변수)

자료형 변환

- ✓ 문자열(str)
  - ✓ str(변수)
- ✓ 리스트(list)
  - ✓ list(변수)
- ✓ 튜플(tuple)
  - ✓ tuple(변수)
- ✓ 집합(set)
  - ✓ set(변수)
- ✓ 딕셔너리(dict)
  - ✓ dict(변수)

자료형 실습

✓ 군집 자료형 실습

파이썬 자료형 5

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓군집 자료형 실습 2

자료형 변환

- ✓ 자료형 변환
  - ✓ 현재 자료형에서 다른 자료형으로 변환
  - ✓ 변수를 다른 자료형으로 변환 완료하려면 변환 후 할당 해야 함. e.g.) a = int(a)
  - ✓ 사용법
    - ✓ 변환할 자료형(변수)
    - ✓ e.g.) int(변수)

파이썬 연산자

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓산술 연산자
✓비교 연산자
✓논리 연산자
✓멤버 연산자
✓연산자 우선순위

산술 연산자

✓산술 연산자

✓산술 계산 시 활용

표. 파이썬의 산술 연산자

산술 연산자	연산
+	더하기
-	빼기
*	곱하기
/	나누기
**	제곱
%	나머지
//	나머지 버리는 나누기

비교 연산자

- ✓비교 연산자
  - ✓값의 크기를 비교하기 위해 사용
  - ✓특정 조건 만족 여부를 판정할 때 사용하며, 조건문, 데이터 추출 등에 유용하게 활용
  - ✓비교 결과는 논리형으로 반환되며, 참일 경우 True, 거짓일 경우 False가 반환됨

비교 연산자

표. 파이썬의 비교 연산자

비교 연산자	연산 (앞의 값을 기준으로 뒤의 값)
>	큰 가?
>=	크거나 같은가?
<	작은가?
<=	작거나 같은가?
==	같은가?
!=	같지 않은가?

논리 연산자

표. 파이썬의 논리 연산자

논리 연산자	연산
and 또는 &	논리곱(AND)
or 또는 |	논리합(OR)
not	논리부정(NOT)

논리 연산자

표. 논리곱(AND) 연산결과

피연산자	연산자	피연산자	결과
True	&	True	True
True	&	False	False
False	&	True	False
False	&	False	False

논리 연산자

표. 논리합(OR) 연산결과

피연산자	연산자	피연산자	결과
True	|	True	True
True	|	False	True
False	|	True	True
False	|	False	False

멤버 연산자

✓멤버 연산자

✓해당 변수2에 변수1이 있는지 없는지 판단

표. 파이썬 멤버 연산자

멤버 연산자	연산
변수1 in 변수2	변수2에 변수1이 있으면 True, 없으면 False
변수1 not in 변수2	변수2에 변수1이 없으면 True, 있을 경우 False

연산자 우선순위

우선순위	연산자
1	(expressions...), [expressions...], {key: value...}, {expressions...}
2	x[index], x[index:index], x(arguments...), x.attribute
3	await x
4	**
5	+x, -x, ~x
6	*, @, /, //, %
7	+, -
8	<<, >>
9	&
10	^
11	|
12	in, not in, is, is not, <, <=, >, >=, !=, ==
13	not x
14	and
15	or
16	if - else
17	lambda
18	:=

출처: https://docs.python.org/3/reference/expressions.html#operator-precedence

파이썬 조건문

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓조건문
  - ✓if
  - ✓else
  - ✓elif

조건문

- ✓ 조건문
  - ✓ 조건에 따라 코드(분석)를 달리 수행해야 할 때
  - ✓ 비교연산자로 조건 만들어 주로 사용됨
  - ✓ 파이썬의 경우 들여쓰기로 조건문 소속 관계 구분(들여쓰기 필수)
  - ✓ if
    - ✓ 조건이 참일 때 특정 코드 수행이 필요할 때
    - ✓ if 조건:

```
#조건이 참일 시
#해당 부분 코드 수행
```

    - ✓ e.g.)

```
a = 5
if a>3:
    print('a는 3보다 크다')
```

조건문

- ✓else
  - ✓조건이 참이 아닐 때 특정 코드 수행이 필요할 때
  - ✓if와 함께 사용

- ✓if 조건:

```
    #조건이 참일 시
    #해당 부분 코드 수행
else:
    #조건이 참이 아닐 때(거짓일 경우)
    #해당 부분 코드 수행
```

조건문

✓e.g.) a = 5

✓ if a>3:

print('a는 3보다 크다')

else:

print('a는 3보다 작거나 같다')

조건문

- ✓elif
  - ✓조건1이 참이 아닐 때 조건2가 참이면 특정 코드 수행이 필요할 때
  - ✓elif 는 1개 이상 추가하여 쓸 수 있음
    - ✓if 조건1:

```
    #조건1이 참일 시 해당 부분 코드 수행
  elif 조건2:
    #조건1이 참 아니면서 조건2가 참일 때 해당 부분 수행
```

조건문

✓if, elif, else 가 모두 있는 경우

✓if 조건1:

#조건1이 참일 시 해당 부분 코드 수행

elif 조건2:

#조건1이 참이 아니면서 조건2가 참일 때 해당 부분 수행

elif 조건3:

#조건1,2가 참이 아니면서 조건3이 참일 때 해당 부분 수행

else:

#위 조건 (조건1,2,3)이 모두 참이 아닐 때 해당 부분 수행

파이썬 반복문

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓반복문
  - ✓for
  - ✓while

반복문

- ✓반복문
  - ✓특정 코드를 반복적으로 수행하기 위해 사용
  - ✓지정한 반복 횟수 또는 주어진 조건이 만족할 때 반복적으로 실행
  - ✓조건문과 마찬가지로 들여쓰기로 소속관계 구분
  - ✓for, while 문이 있음
  - ✓break, continue, pass는 반복문 종료 등 반복문 내의 제어를 위한 명령어

반복문

- ✓ for 문
  - ✓ 지정한 횟수만큼 특정 코드를 반복적으로 수행해야 할 때 유용함
  - ✓ for 변수1 in 변수2:

    #해당 부분 코드 반복 수행
  - ✓ 내장 함수 range와 자주 함께 활용됨
    - ✓ range(a,b+1)
      - ✓ a부터 시작하여 b까지 1씩 증가하는 리스트 생성
    - ✓ range(b+1)
      - ✓ 0부터 시작하여 b까지 1씩 증가하는 리스트 생성

  - ✓ e.g.)

```
for i in range(1,11):
    print(i)
```

반복문

✓E.g.) for i in range(1,11)

print(i)

표. for 문 변수 값 변화

회차	range (1,11)	1	2	3	4	5	6	7	8	9	10	출력
1	변수(i)											
2	변수(i)											
3	변수(i)											
4	변수(i)											
5	변수(i)											
6	변수(i)											
7	변수(i)											
8	변수(i)											
9	변수(i)											
10	변수(i)											

반복문

✓E.g.) for i in range(1,11)

print(i)

표. for 문 변수 값 변화

회차	range (1,11)	1	2	3	4	5	6	7	8	9	10	출력
1	변수(i)	i=1										1
2	변수(i)		i=2									2
3	변수(i)			i=3								3
4	변수(i)				i=4							4
5	변수(i)					i=5						5
6	변수(i)						i=6					6
7	변수(i)							i=7				7
8	변수(i)								i=8			8
9	변수(i)									i=9		9
10	변수(i)										i=10	10

반복문

✓e.g.1)
```
for i in range(1,11):
    print(i)
```

e.g.2)
```
for i in [1,2,3,4,5,6,7,8,9,10]:
    print(i)
```

e.g.3)
```
a = [1,2,3,4,5,6,7,8,9,10]
for i in range(len(a)):
    print(a[i])
```

반복문

- ✓while 문
  - ✓주어진 조건이 만족할 때 반복적으로 수행
  - ✓조건을 적절히 주지 않으면 무한히 반복될 수 있음
  - ✓무한히 반복되는 코드가 아니라면, 반복문 안에 조건을 변화시킬 수 있는 코드를 넣어줄 것
  - ✓while문 전에 초기 값을 설정

- ✓while 조건:

  #해당 부분 코드 반복 수행

반복문

✓E.g.)

```
i = 1
while i <= 10:
    print(i)
    i = i + 1
```

표. while 문 변수 값 변화

i											
조건 (i<=10)											
수행 회차											
출력											
i 값											

반복문

✓E.g.)

```
i = 1
while i <= 10:
    print(i)
    i = i + 1
```

표. while 문 변수 값 변화

i	1	2	3	4	5	6	7	8	9	10	11
조건 (i<=10)	T	T	T	T	T	T	T	T	T	T	F
수행 회차	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**X**
출력	1	2	3	4	5	6	7	8	9	10	
i 값	2	3	4	5	6	7	8	9	10	11	

정리

- ✓반복문
  - ✓for
  - ✓while

파이썬 반복문 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓반복문 복습
- ✓반복문 제어
  - ✓break
  - ✓continue
  - ✓pass

반복문

- ✓반복문
  - ✓특정 코드를 반복적으로 수행하기 위해 사용
  - ✓지정한 반복 횟수 또는 주어진 조건이 만족할 때 반복적으로 실행
  - ✓조건문과 마찬가지로 들여쓰기로 소속관계 구분
  - ✓for, while 문이 있음
  - ✓break, continue, pass는 반복문 종료 등 반복문 내의 제어를 위한 명령어

반복문

- ✓ for 문
  - ✓ 지정한 횟수만큼 특정 코드를 반복적으로 수행해야 할 때 유용함
  - ✓ for 변수1 in 변수2:

    #해당 부분 코드 반복 수행
  - ✓ 내장 함수 range와 자주 함께 활용됨
    - ✓ range(a,b+1)
      - ✓ a부터 시작하여 b까지 1씩 증가하는 리스트 생성
    - ✓ range(b+1)
      - ✓ 0부터 시작하여 b까지 1씩 증가하는 리스트 생성

  - ✓ e.g.)

```
for i in range(1,11):
    print(i)
```

반복문

✓E.g.) for i in range(1,11)

print(i)

표. for 문 변수 값 변화

회차	range (1,11)	1	2	3	4	5	6	7	8	9	10	출력
1	변수(i)	i=1										1
2	변수(i)		i=2									2
3	변수(i)			i=3								3
4	변수(i)				i=4							4
5	변수(i)					i=5						5
6	변수(i)						i=6					6
7	변수(i)							i=7				7
8	변수(i)								i=8			8
9	변수(i)									i=9		9
10	변수(i)										i=10	10

반복문

✓e.g.1)
```
for i in range(1,11):
    print(i)
```

e.g.2)
```
for i in [1,2,3,4,5,6,7,8,9,10]:
    print(i)
```

e.g.3)
```
a = [1,2,3,4,5,6,7,8,9,10]
for i in range(len(a)):
    print(a[i])
```

반복문

- ✓while 문
  - ✓주어진 조건이 만족할 때 반복적으로 수행
  - ✓조건을 적절히 주지 않으면 무한히 반복될 수 있음
  - ✓무한히 반복되는 코드가 아니라면, 반복문 안에 조건을 변화시킬 수 있는 코드를 넣어줄 것
  - ✓while문 전에 초기 값을 설정

- ✓while 조건:

  #해당 부분 코드 반복 수행

반복문

✓E.g.)

```
i = 1
while i <= 10:
    print(i)
    i = i + 1
```

표. while 문 변수 값 변화

i	1	2	3	4	5	6	7	8	9	10	11
조건 (i<=10)	T	T	T	T	T	T	T	T	T	T	F
수행 회차	**1**	**2**	**3**	**4**	**5**	**6**	**7**	**8**	**9**	**10**	**X**
출력	1	2	3	4	5	6	7	8	9	10	
i 값	2	3	4	5	6	7	8	9	10	11	

break

- ✓break
  - ✓가장 가깝게 속한 반복문 하나를 빠져나옴
  - ✓e.g.1)
  - ✓while 조건1:

```
    if 조건2:
        break
```

break

✓e.g.2)

✓

```
for i in range(1,11):
    print(i)
    if i%3==0:
        break
```

break

✓e.g.2)

```
for i in range(1,11):
    print(i)
    if i%3==0:
        break
```

Ans)

✓1

✓2

✓3

continue

- ✓continue
  - ✓가장 마지막으로 속한 반복문 내에서 continue 이후의 코드는 실행하지 않고, 해당 반복문의 다음 반복으로 넘어 감
  - ✓e.g.1)
  - ✓while 조건1:

```
    if 조건2:
        continue
    print('a')
```

  - ✓while 조건1:

```
    if 조건2:
        #continue
    print('a')
```

continue

✓e.g.2)

```
i = 1
while i <= 10:
    i = i + 1
    if i%3==0:
        continue
    print(i)
```

continue

✓e.g.2)

```
i = 1
while i <= 10:
    i = i + 1
    if i%3==0:
        continue
    print(i)
```

Ans)

✓2

✓4

✓5

✓7

✓8

✓10

✓11

pass

- ✓ pass
  - ✓ 변함 없음
  - ✓ 변함 없음을 표현하기 위해 사용
  - ✓ 조건문, 함수 등에서도 활용가능

  - ✓ e.g.1)
  - ✓ i = 1
  - ✓ while i <= 10:

```
    i = i + 1
    if i%3==0:
        pass
    print(i)
```

pass

- ✓ pass
  - ✓ 변함 없음
  - ✓ 변함 없음을 표현하기 위해 사용
  - ✓ 조건문, 함수 등에서도 활용가능

  - ✓ e.g.1)
  - ✓ i = 1
  - ✓ while i <= 10:

```
    i = i + 1
    if i%3==0:
        pass
    print(i)
```

Ans)

- ✓ 2
- ✓ 3
- ✓ 4
- ✓ 5
- ✓ 6
- ✓ 7
- ✓ 8
- ✓ 9
- ✓ 10
- ✓ 11

파이썬 함수

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓함수

함수

- ✓함수
  - ✓연산을 수행하는 객체화 된 코드
  - ✓자주 사용하는 연산을 객체화 시키기에 유용
  - ✓매개변수가 있는 혹은 없는, 그리고 리턴(반환)이 있는 혹은 없는 함수가 있음
  - ✓간결하고 생산성을 높일 수 있음
  - ✓파이썬 함수 리턴 값은 하나의 튜플(tuple)로 반환됨
  - ✓return을 만나면 결과 값을 리턴하고 함수를 종료함

함수

✓사용법

✓함수 정의

✓def 함수명(매개변수):

return 리턴 값

✓정의 후 함수 사용

✓함수명(인수)

✓인수는 순서에 맞게 인수를 넣어 사용하거나,

순서 관계없이 매개변수명을 직접 기입하여(e.g. a=1, b=2) 사용

* 매개변수(parameter), 인수(argument)

함수

(입력 값 할당 코드)

연산

(결과 코드)

그림. 함수화 되지 않은 일반 분석 코드

함수

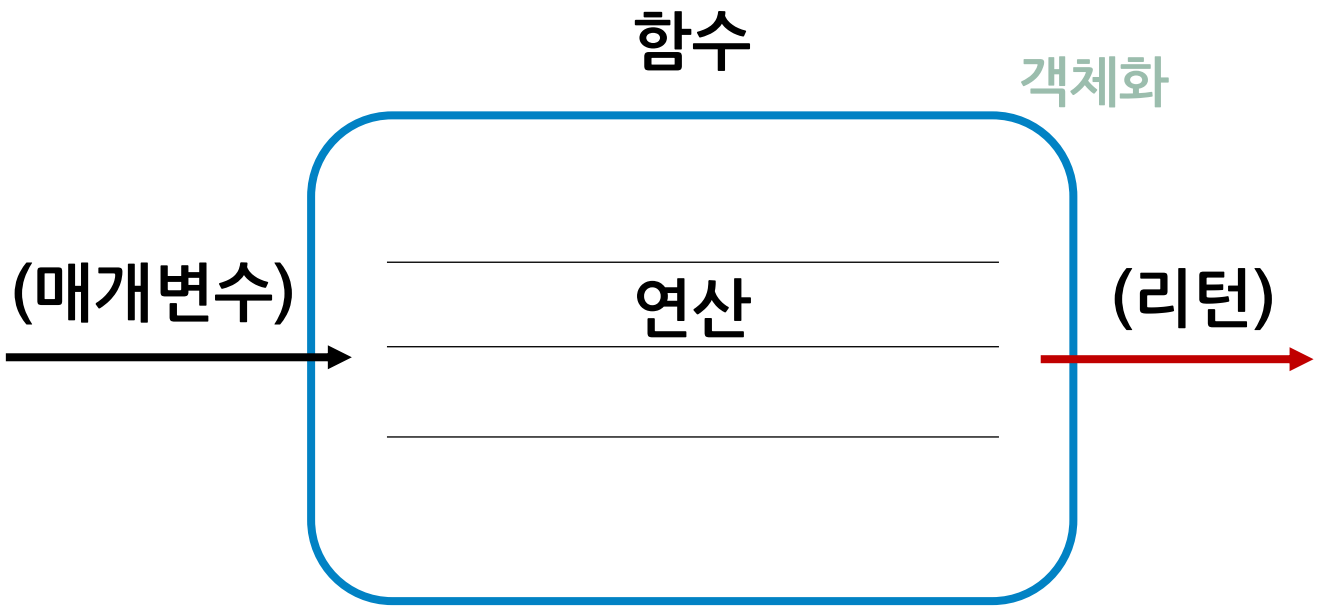


그림. 함수 개념

✓함수화 된 코드의 장점

(입력 값 a 할당 코드)

연산

(결과 코드)

(입력 값 b 할당 코드)

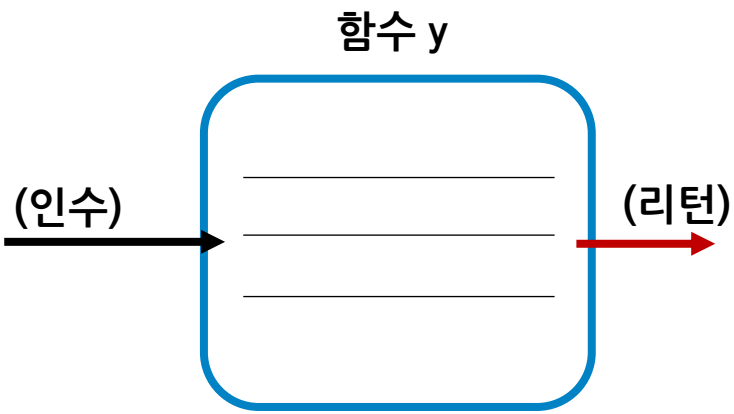
연산

(결과 코드)

(입력 값 c 할당 코드)

연산

(결과 코드)



y(a)

y(b)

y(b)

함수

✓매개변수, 리턴

✓매개변수가 있는 / 없는, 리턴이 있는 / 없는 함수가 있음

✓리턴이 있는 경우 함수 결과가 리턴 값 자체가 되어 버림

✓리턴 시 함수 종료

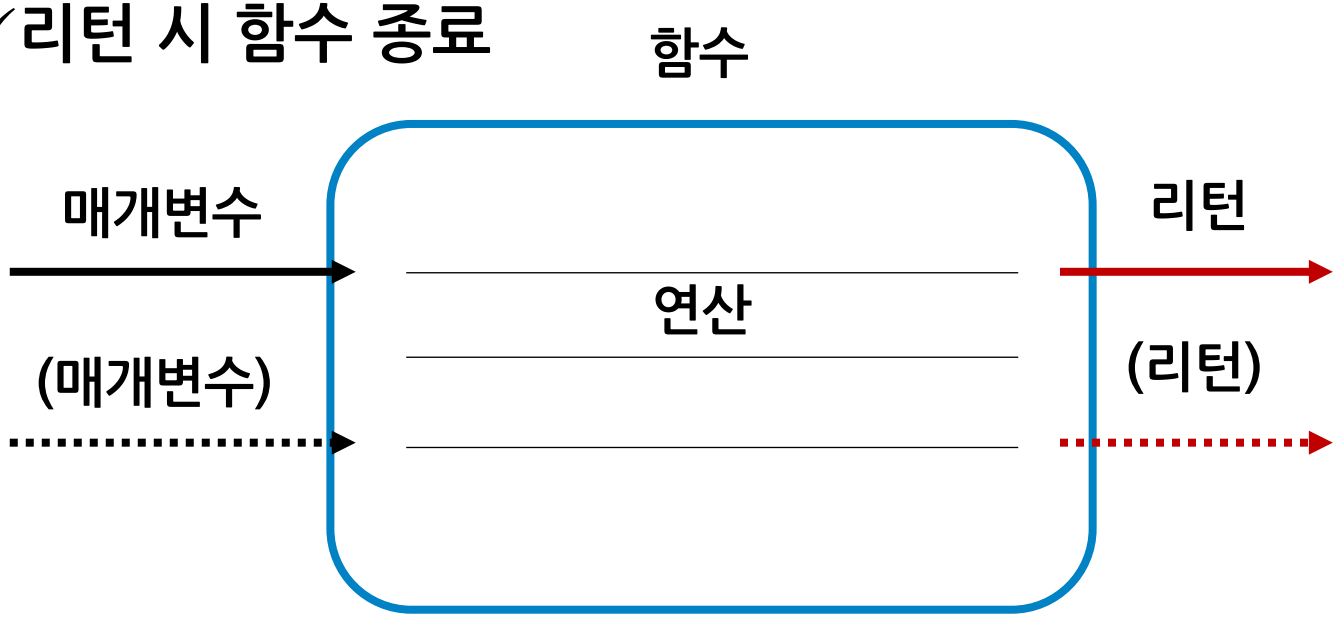


그림. 함수 개념

함수

- ✓리턴이 있는 경우 함수 결과가 리턴 값 자체가 되어 버림
  - ✓e.g.) 매개변수를 3배 곱하여 리턴하는 함수 y
  - ✓a = y(5)
  - ✓print(a)
  - ✓15
  - ✓print(type(a))
  - ✓class int

파이썬 함수 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓매개변수 초기값

✓다수 연산 값 리턴

✓가변 매개변수

✓지역변수, 전역변수

✓람다(lambda) 함수

함수 복습

- ✓함수
  - ✓연산을 수행하는 객체화 된 코드
  - ✓자주 사용하는 연산을 객체화 시키기에 유용
  - ✓매개변수가 있는 혹은 없는, 그리고 리턴(반환)이 있는 혹은 없는 함수가 있음
  - ✓간결하고 생산성을 높일 수 있음
  - ✓파이썬 함수 리턴 값은 하나의 튜플(tuple)로 반환됨
  - ✓return을 만나면 결과 값을 리턴하고 함수를 종료함

함수 복습

✓사용법

✓함수 정의

✓def 함수명(매개변수):

return 리턴 값

✓정의 후 함수 사용

✓함수명(인수)

✓위치에 맞게 인수를 넣어 사용

* 매개변수(parameter), 인수(argument)

매개변수 초기값

- ✓매개변수 초기값
  - ✓함수 정의 시 매개변수의 초기값을 미리 설정하는 것이 가능
  - ✓함수 사용시 인수를 입력하지 않으면 함수 정의 시 초기값으로 인수가 설정됨
  - ✓사용법
    - ✓def 함수명(매개변수=초기값):

```
    # return 리턴 값
```

다수 연산 값 리턴

- ✓다수 연산 값 리턴
  - ✓여러 값이 있는 하나의 튜플(tuple)로 리턴
  - ✓사용법
    - ✓def 함수명(매개변수):

      return 값1, 값2, …
    - ✓변수1, 변수2, … = 함수(인수) 또는
    - ✓변수 = 함수(인수)

가변 매개변수

- ✓사용법
- ✓함수 정의
  - ✓def 함수명(매개변수):

    # return 리턴 값

- ✓가변 매개변수
  - ✓매개변수 개수가 정해지지 않은 함수의 경우 매개변수 앞에 *를 사용하여 가변의 매개변수를 받음
  - ✓def 함수명(*매개변수):
  - ✓매개변수 명은 일반적으로 *args 를 많이 사용함

지역변수, 전역변수

- ✓지역변수(local variable)
  - ✓함수에서 활용하는 변수는 함수에서만 독립적으로 활용 가능하며 이를 지역변수(local variable)라 함

- ✓전역변수(global variable)
  - ✓함수 안에서 함수 밖의 변수를 다루고자 할 때 활용하며 변수명 앞에 global 을 표기함
  - ✓함수는 독립적으로 동작하는 것이 가장 이상적이므로 전역변수는 가급적 지양

람다(lambda) 함수

- ✓람다(lambda) 함수
  - ✓사용 빈도가 낮고, 간결한 연산의 함수 표현 시 활용법
  - ✓리턴 표현이 없지만 리턴 함
  - ✓사용법
    - ✓함수명 = lambda 매개변수 : 리턴값

- ✓e.g.) mul = lambda a, b: a*b

  result = mul(2,3)

파이썬 조건문 반복문 응용

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓함수2 (계속)
- ✓조건문 반복문 응용

조건문 반복문 응용

✓[기본 1] 1부터 100까지의 숫자 중 홀수만 출력하는 프로그램을 작성하시오.

✓[기본 2] 1단부터 9단까지 순서대로 구구단을 출력하는 프로그램을 작성하시오.

✓[기본 3] 9단부터 1단까지 순서로 구구단을 출력하는 프로그램을 작성하시오. (앞의 숫자만 역순, 뒤의 숫자는 1~9 순서대로 출력)

✓[기본 4] 1단부터 9단까지 구구단 중 2단, 5단을 제외하고 출력하는 프로그램을 작성하시오. (앞자리에 오는 숫자 만을 기준으로 제외)

조건문 반복문 응용

✓[기본 5] 다음을 “*”하나와 반복문을 활용하여 아래의 별을 출력하는 프로그램을 작성하시오.

```
*
**
***
****
*****
```

조건문 반복문 응용

✓[기본 6] 다음을 “*”하나와 반복문을 활용하여 아래의 별을 출력하는 프로그램을 작성하시오.

```
*****
****
***
**
*
```

파이썬 조건문 반복문 응용 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓조건문 반복문 응용2

조건문 반복문 응용

✓[기본 5] 다음을 “*”하나와 반복문을 활용하여 아래의 별을 출력하는 프로그램을 작성하시오.

```
*
**
***
****
*****
```

조건문 반복문 응용

표. 기본 5 * 문제 i, j에 따른 출력 변화

j range (1,i+1) / i range(1,6)	1	2	3	4	5	
1	*	'\n'				
2	*	*	'\n'			
3	*	*	*	'\n'		
4	*	*	*	*	'\n'	
5	*	*	*	*	*	'\n'

조건문 반복문 응용

표. 기본 5 * 문제 i, j에 따른 출력 변화

j range (1,i+1) / i range(1,6)	1	2	3	4	5	
1	*	'\n'				
2	*	*	'\n'			
3	*	*	*	'\n'		
4	*	*	*	*	'\n'	
5	*	*	*	*	*	'\n'

기본 5 문제 풀이 코드

```
for i in range(1,6):
    for j in range(1,i+1):
        print('*', end='')
    print('')
```

조건문 반복문 응용

표. 기본 5 * 문제 i, j에 따른 출력 변화

i / range(1,6)	1	2	3	4	5	
1	*	'\n'				
2	*	*	'\n'			
3	*	*	*	'\n'		
4	*	*	*	*	'\n'	
5	*	*	*	*	*	'\n'

기본 5 문제 풀이 코드

```
for i in range(1,6):
    print('*' * i)
```

조건문 반복문 응용

✓[기본 7] 두 양의 정수 a와 b가 주어질 때 두 수의 공약수를 출력하시오. (a는 12, b는 6으로 주어진다고 하자)

✓[기본 8] 두 양의 정수 a와 b가 주어질 때 두 수의 최대공약수를 출력하시오. (a는 12, b는 6으로 주어진다고 하자)

✓[기본 9] 두 양의 정수 a와 b가 주어질 때 두 수의 최소공배수를 출력하시오. (a는 12, b는 6으로 주어진다고 하자)

✓[기본 10] 2부터 100까지의 정수 중 소수를 출력하시오. (소수는 1과 자기 자신 외에 나누어 떨어지는 수가 없는 1보다 큰 양의 정수)

파이썬 함수 응용

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓함수 응용

함수 응용

✓ [함수 1] 두 양의 정수를 매개 변수로 받아서 두 수의 최대공약수를 리턴하는 함수를 만드시오. # [기본 8] 두 양의 정수 a와 b가 주어질 때 두 수의 최대공약수를 출력하시오. (a는 12, b는 6으로 주어진다고 하자)

✓ [함수 2] 두 양의 정수를 매개 변수로 받아서 두 수의 최대공약수를 출력하시는 함수를 만드시오.

✓ [함수 3] 두 양의 정수를 매개 변수로 받아서 두 수의 최소공배수를 리턴하는 함수를 만드시오. #[기본 9] 두 양의 정수 a와 b가 주어질 때 두 수의 최소공배수를 출력하시오. (a는 12, b는 6으로 주어진다고 하자)

함수 응용

✓[함수 4] 두 양의 정수를 매개 변수로 받아서 두 수의 최소공배수를 출력하는 함수를 만드시오.

✓[함수 5] 2 이상의 양의 정수 하나를 입력 받아서, 2부터 해당 정수까지의 정수 중 소수를 출력하는 함수를 만드시오. # [기본 10] 2부터 100까지의 정수 중 소수를 출력하시오. (소수는 1과 자기 자신 외에 나누어 떨어지는 수가 없는 1보다 큰 양의 정수)

파이썬 입출력

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓입출력

입력

- ✓입력 (input)
  - ✓내장함수 input( ) 를 통해 입력을 받을 수 있음
  - ✓파일을 통해 입력을 받을 수 있음
    - ✓open, read, close 메소드 등을 활용 가능

- ✓출력 (output)
  - ✓print( ) 등을 통해 결과를 출력할 수 있음
  - ✓파일을 통해 결과를 출력할 수 있음
    - ✓open, write, close 메소드 등을 활용 가능

파일 출력

✓파이썬에서 open() 함수를 사용하여 파일을 열 때, 두 번째 인자로 . 'r' (읽기)과 'w' (쓰기) 등 파일을 어떤 목적으로 열 것인지를 결정하는 파일 열기 모드(mode)를 지정해야 함

파일 출력

모드	설명	파일이 없을 때	파일이 있을 때
'r'	**읽기 모드 (Read)**: 파일의 내용을 읽기만 할 때 사용	FileNotFoundError 발생	파일의 처음부터 내용을 읽습니다.
'w'	**쓰기 모드 (Write)**: 파일에 새로운 내용을 쓸 때 사용	새로운 파일을 생성합니다.	**기존 내용을 모두 삭제**하고 새로 씁니다.
'a'	**추가 모드 (Append)**: 파일의 마지막에 새로운 내용을 추가할 때 사용	새로운 파일을 생성합니다.	기존 내용의 맨 끝에 이어서 씁니다.
'x'	**배타적 생성 모드 (Exclusive Creation)**: 파일이 없는 경우에만 새로 생성하여 쓸 때 사용	새로운 파일을 생성합니다.	FileExistsError 발생

파일 출력

✓open() 함수는 파일 모드 외에도 다른 파라메터를 가질 수 있으며 encoding 등이 대표적임

✓encoding: 파일의 인코딩 방식을 지정합니다. 텍스트 파일을 읽거나 쓸 때, 운영체제의 기본 인코딩 방식과 파일의 인코딩 방식이 다를 경우 글자가 깨질 수 있습니다. 이때 encoding을 명시해주면 문제를 해결할 수 있음

✓UTF-8 : 가장 널리 사용, CP949 : 엑셀에 주로 사용

✓file = open('myfile.txt', 'r', encoding='utf-8')

입력문 응용

✓[입력문 1] 사용자에게 두 양의 정수를 입력 받고, 두 수의 최대 공약수를 출력 하시오.

✓[입력문 2] 사용자에게 두 양의 정수를 입력 받고, 두 수의 최소 공배수를 출력 하시오.

✓[입력문 3] 사용자에게 하나의 양의 정수 n을 입력 받고 2부터 n까지의 정수 중 소수를 출력 하시오. (소수는 1과 자기 자신 외에 나누어 떨어지는 수가 없는 1보다 큰 양의 정수)

입력문 응용

✓[입력문 4] 사용자에게 하나의 양의 정수 n을 입력 받고 2부터 n까지의 정수 중 소수를 리턴하는 prime 함수를 만드시오.

✓[입력문 5] 사용자에게 입력 받은 양의 정수가 소수인지 아닌지를 출력하는 prime_tester 함수를 만드시오.

파이썬 입출력 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓입력문 응용

파이썬 스타일 코드

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓파이썬 스타일 코드 (pythonic code)
- ✓리스트 컴프리헨션 (list comprehension)
- ✓enumerate

파이썬 스타일 코드

- ✓파이썬 스타일 코드 (Pythonic code)
  - ✓파이썬에서 활용되는 간결한 문법의 코드
  - ✓생산성 향상
  - ✓list comprehension, enumerate, split, join, zip

리스트 컴프리헨션 (list comprehension)

- ✓리스트 컴프리헨션 (list comprehension)
  - ✓리스트 내에서 조건문, 반복문 활용
  - ✓사용법1
    - ✓[연산 for 변수1 in 변수2]
    - ✓e.g.) square = [i**2 for i in [1,2,3]]

리스트 컴프리헨션 (list comprehension)

- ✓리스트 컴프리헨션 (list comprehension)
  - ✓리스트 내에서 조건문, 반복문 활용
  - ✓사용법2
    - ✓[연산 for 변수1 in 변수2 if 조건]
    - ✓e.g.) square = [i**2 for i in [1,2,3] if i%3==0]

enumerate

- ✓enumerate
  - ✓인덱스와 값을 한 쌍으로 반환
  - ✓for 문에서 인덱스와 값을 함께 다룰 때 range, len 함수 등을 대체하여 간결한 코드 작성에 용이함
  - ✓사용법
    - ✓e.g.)
    - ✓for i, num in enumerate([1,2,3]):
      - ✓print('%d, %d'%(i,num))

파이썬 스타일 코드 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓리스트 컴프리헨션 응용
- ✓enumerate 응용

리스트 컴프리헨션 (list comprehension)

- ✓리스트 컴프리헨션 (list comprehension)
  - ✓리스트 내에서 조건문, 반복문 활용
  - ✓사용법1
    - ✓[연산 for 변수1 in 변수2]
    - ✓e.g.) square = [i**2 for i in [1,2,3]]

리스트 컴프리헨션 (list comprehension)

- ✓리스트 컴프리헨션 (list comprehension)
  - ✓리스트 내에서 조건문, 반복문 활용
  - ✓사용법2
    - ✓[연산 for 변수1 in 변수2 if 조건]
    - ✓e.g.) square = [i**2 for i in [1,2,3] if i%3==0]

enumerate

- ✓enumerate
  - ✓인덱스와 값을 한 쌍으로 반환
  - ✓for 문에서 인덱스와 값을 함께 다룰 때 range, len 함수 등을 대체하여 간결한 코드 작성에 용이함
  - ✓사용법
    - ✓e.g.)
    - ✓for i, num in enumerate([1,2,3]):
      - ✓print('%d, %d'%(i,num))

파이썬 스타일 코드

✓ 리스트 컴프리헨션을 활용하여 아래의 프로그램을 작성하시오.

✓ [응용 1] 1부터 100까지의 숫자 중 홀수만 출력하는 프로그램을 작성하시오.

✓ [응용 2] 1단부터 9단까지 구구단을 2차원 배열 형태의 리스트로 담으시오.

✓ [응용 3] 9단부터 1단까지 구구단을 2차원 배열 형태의 리스트로 담으시오.

(앞의 숫자만 역순, 뒤의 숫자는 1~9 순서대로)

✓ [응용 4] 1단부터 9단까지 구구단 중 2단, 5단을 제외하고 2차원 배열 형태의 리스트로 담는 프로그램을 작성하시오. (앞자리에 오는 숫자 만을 기준으로 제외)

파이썬 스타일 코드

✓ [응용 5] 두 양의 정수 a와 b가 주어질 때 두 수의 공약수를 출력하시오. (a=12, b=6. 응용 6, 7 동일)

✓ [응용 6] 두 양의 정수 a와 b가 주어질 때 두 수의 최대공약수를 출력하시오.

✓ [응용 7] 두 양의 정수 a와 b가 주어질 때 두 수의 최소공배수를 출력하시오.

✓ 리스트 컴프리헨션, enumerate를 활용하여 아래의 프로그램을 작성하시오.

✓ [응용 8] 2부터 100까지의 정수 중 소수가 아닌 수를 찾아 작은 순서대로 1번부터 번호를 매겨 번호와 함께 숫자를 출력 하시오. (소수는 1과 자기 자신 외에 나누어 떨어지는 수가 없는 1보다 큰 양의 정수)

파이썬 스타일 코드 3

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓리스트 컴프리헨션 응용
- ✓enumerate 응용

클래스

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓클래스

클래스

- ✓클래스(class)
  - ✓객체(object) 생성이 가능한 설계도
  - ✓클래스를 바탕으로 해당 클래스의 객체(object)인 인스턴스(instance)를 생성할 수 있음

클래스

- ✓메소드 (methods), 변수 (variable)로 구성
  - ✓메소드: 클래스에서 정의된 함수
  - ✓변수:
    - ✓객체 변수 (주로 활용, 객체 별 독립)
    - ✓클래스 변수 (해당 클래스의 객체 전체가 공유)

클래스

✓클래스 정의

✓class 클래스명:

*첫 글자 대문자 추천

✓def __init__(self, 매개변수):

✓self.매개변수 = 매개변수

#__init__ 메소드는 생성자

✓def 메소드명(self):

✓# 메소드 정의

✓클래스변수명 = 변수값

클래스

- ✓객체 생성
  - ✓객체명 = 클래스명(매개변수)
  - ✓객체명.메소드명( ) # 메소드 활용

- ✓클래스변수 사용법
  - ✓클래스명.클래스변수
  - ✓객체명.클래스변수

클래스

- ✓상속(Inheritance)
  - ✓B 클래스가 A 클래스로부터 상속 받았다면, A 클래스를 부모 클래스(parent class) 또는 슈퍼 클래스(super class), B 클래스를 자식 클래스(child class) 또는 서브 클래스(subclass) 라 함
  - ✓자식 클래스는 부모 클래스의 메소드와 변수를 받아 사용
  - ✓자식 클래스에서 부모 클래스에서 사용한 메소드 명을 다시 정의하여 사용하는 경우 오버라이딩(overriding)이라고 하며 해당 메소드 사용시 자식 클래스에서 정의된 메소드가 사용됨
  - ✓사용법
    - ✓class 클래스명(상속 받을 클래스명):

```
    # 클래스 정의
```

클래스 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

- ✓클래스 복습
- ✓상속
- ✓클래스 구현

클래스

- ✓클래스(class)
  - ✓객체(object) 생성이 가능한 설계도
  - ✓클래스를 바탕으로 해당 클래스의 객체(object)인 인스턴스(instance)를 생성할 수 있음

클래스

- ✓메소드(methods), 변수(variable)로 구성
  - ✓메소드: 클래스에서 정의된 함수
  - ✓변수: 객체 변수(주로 활용, 객체 별 독립), 클래스 변수(해당 클래스의 객체 전체가 공유함)

클래스

✓클래스 정의

✓class 클래스명:

*첫 글자 대문자 추천

✓def __init__(self, 매개변수):

✓self.매개변수 = 매개변수

#__init__ 메소드는 생성자

✓def 메소드명(self):

✓# 메소드 정의

✓클래스변수명 = 변수값

클래스

- ✓객체 생성
  - ✓객체명 = 클래스명(매개변수)
  - ✓객체명.메소드명( ) # 메소드 활용

- ✓클래스변수 사용법
  - ✓클래스명.클래스변수
  - ✓객체명.클래스변수

클래스

- ✓상속(Inheritance)
  - ✓B 클래스가 A 클래스로부터 상속 받았다면, A 클래스를 부모 클래스(parent class) 또는 슈퍼 클래스(super class), B 클래스를 자식 클래스(child class) 또는 서브 클래스(subclass) 라 함
  - ✓자식 클래스는 부모 클래스의 메소드와 변수를 받아 사용
  - ✓자식 클래스에서 부모 클래스에서 사용한 메소드 명을 다시 정의하여 사용하는 경우 오버라이딩(overriding)이라고 하며 자식 객체에서 해당 메소드 사용시 자식 클래스에서 정의된 메소드가 사용됨
  - ✓사용법
    - ✓class 클래스명(상속 받을 클래스명):

```
# 클래스 정의
```

클래스 구현

- ✓[클래스 1]
  - ✓클래스명: GcdLcm
  - ✓매개변수: 양의 정수 2개
  - ✓메소드:
    - ✓gcd
      - ✓두 수의 최대공약수 리턴
    - ✓lcm
      - ✓두 수의 최소공배수 리턴

모듈 및 패키지

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓모듈 (module)

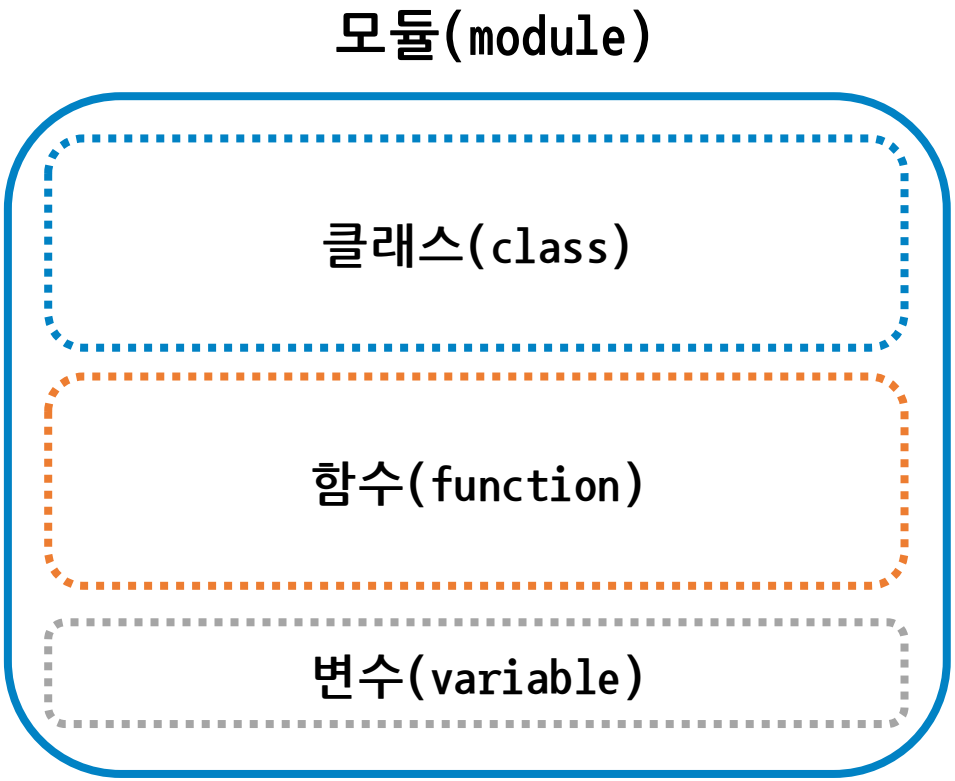
모듈

- ✓모듈 (module)
  - ✓클래스, 함수, 변수 등이 포함될 수 있는 파이썬 파일

모듈

✓모듈 (module)

✓클래스, 함수, 변수 등이 포함될 수 있는 파이썬 파일



모듈

- ✓종류
  - ✓표준
  - ✓서드 파티
  - ✓사용자 정의

모듈

✓사용법

✓모듈인 파이썬 파일(.py)을 같은 폴더에 둘 것

✓주피터 노트북 파일(.ipynb)은 .py 확장자로 변환 필요

```
!jupyter nbconvert  - to script 파일명.ipynb
```

✓import 모듈명 # import는 첫 줄에 작성 추천

✓모듈명.모듈함수명()

✓모듈명.변수명

✓객체 = 모듈명.클래스명()

모듈

- ✓모듈명을 간략히 하고자 별칭(alias) 사용 가능
- ✓numpy의 경우 np, pandas의 경우 pd 등으로 주로 사용됨

- ✓import 모듈명 as 별칭

- ✓별칭.모듈함수명()
- ✓별칭.변수명
- ✓객체 = 별칭.클래스명()

모듈

모듈 함수/클래스를 바로 활용하고자 할 때

✓from 모듈명 import 모듈함수 또는 클래스

✓모듈함수명()

✓모듈클래스명()

해당 모듈의 모든 함수 임포트

✓from 모듈명 import *

모듈

✓사용법

✓if __name__==‘__main__’:

임포트 시에는 실행하지 않고

모듈 파일 직접 실행 시만 실행

모듈

✓[모듈 및 패키지 1]

✓1. [클래스 1] 문제의 GcdLcm 클래스를 가지고 있는 gcdlcm 모듈을 만들고, 해당 모듈을 활용하여 12와 6의 최대공약수와 최소공배수를 구하시오.

모듈 및 패키지 2

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓패키지 (package)

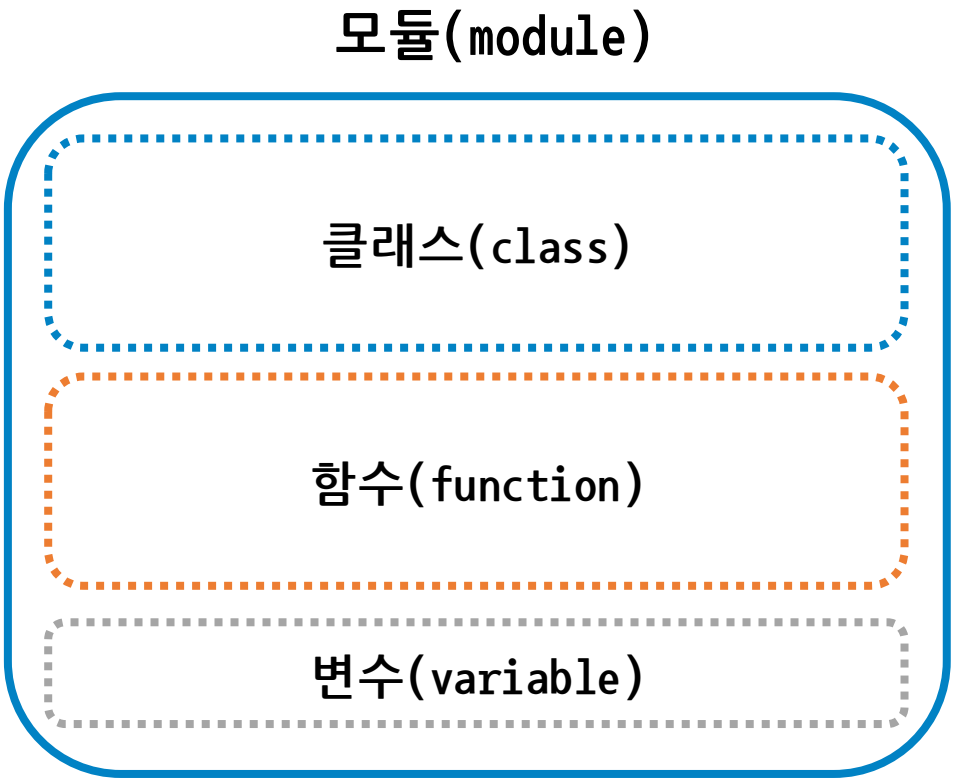
모듈

- ✓모듈 (module)
  - ✓클래스, 함수, 변수 등이 포함될 수 있는 파이썬 파일

모듈

✓모듈 (module)

✓클래스, 함수, 변수 등이 포함될 수 있는 파이썬 파일



모듈

- ✓종류
  - ✓표준
  - ✓서드 파티
  - ✓사용자 정의

모듈

✓사용법

✓모듈인 파이썬 파일(.py)을 같은 폴더에 둘 것

✓주피터 노트북 파일(.ipynb)은 .py 확장자로 변환 필요

```
!jupyter nbconvert  - to script 파일명.ipynb
```

✓import 모듈명 # import는 첫 줄에 작성 추천

✓모듈명.모듈함수명()

✓모듈명.변수명

✓객체 = 모듈명.클래스명()

모듈

- ✓모듈명을 간략히 하고자 별칭(alias) 사용 가능
- ✓numpy의 경우 np, pandas의 경우 pd 등으로 주로 사용됨

- ✓import 모듈명 as 별칭

- ✓별칭.모듈함수명()
- ✓별칭.변수명
- ✓객체 = 별칭.클래스명()

모듈

모듈 함수/클래스를 바로 활용하고자 할 때

✓from 모듈명 import 모듈함수 또는 클래스

✓모듈함수명()

✓모듈클래스명()

해당 모듈의 모든 함수 임포트

✓from 모듈명 import *

모듈

✓사용법

✓if __name__==‘__main__’:

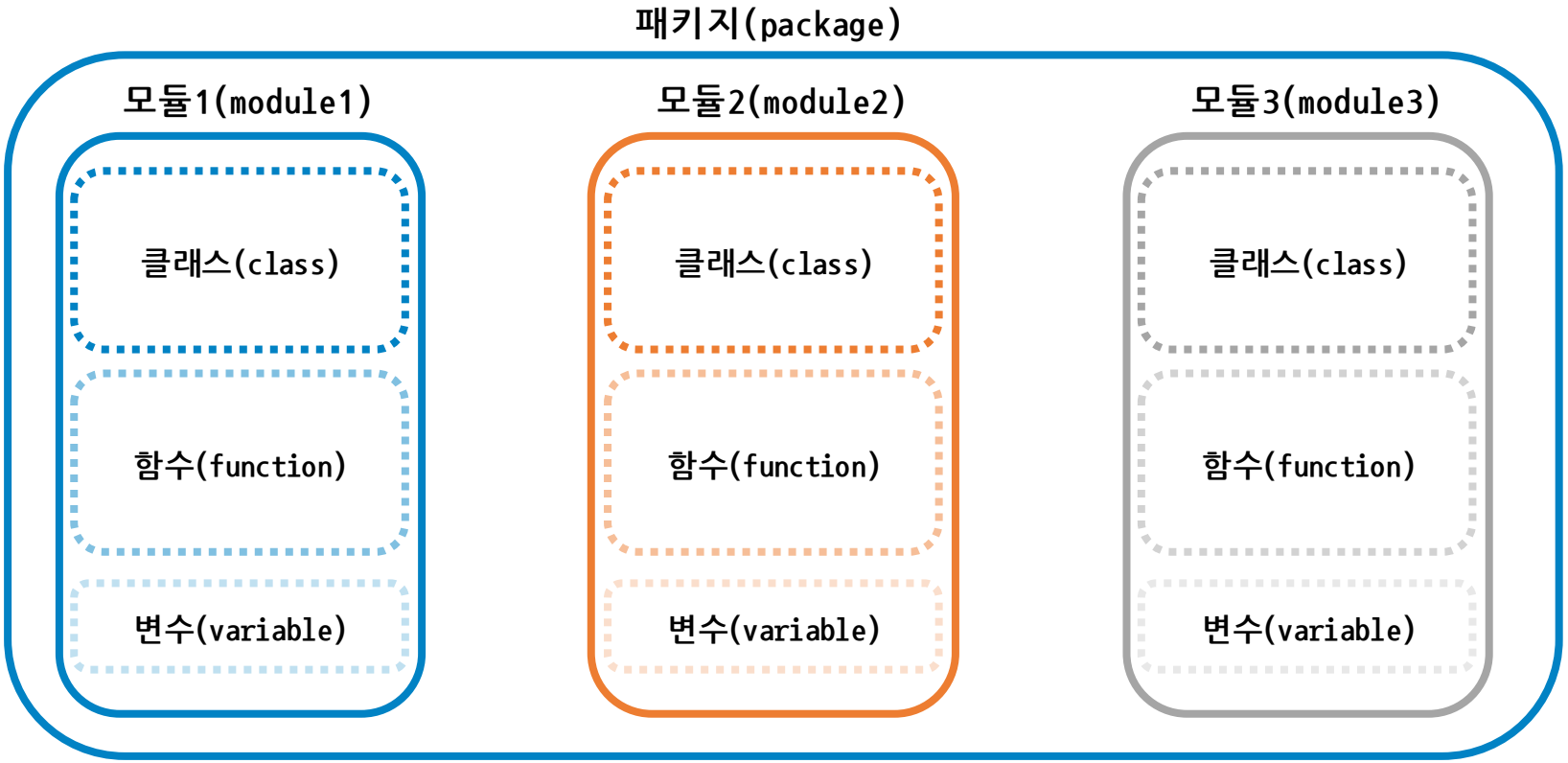
임포트 시에는 실행하지 않고

모듈 파일 직접 실행 시만 실행

패키지

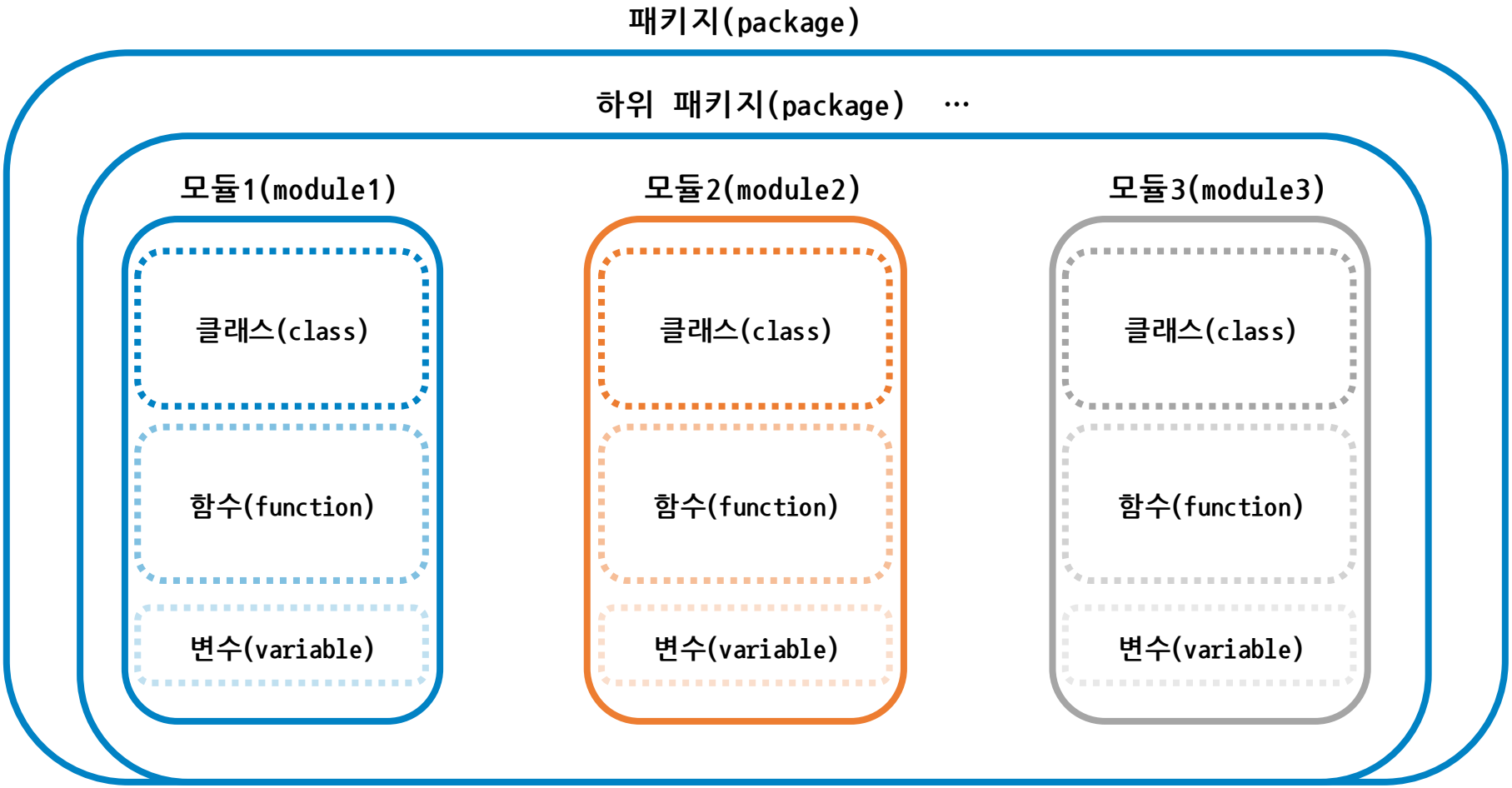
✓패키지 (package)

✓모듈 및 하위 패키지에 대해서 디렉토리(directory) 형태의 계층적 관리가 가능한 묶음



패키지

✓모듈 및 하위 패키지에 대해서 디렉토리(directory) 형태의 계층적 관리가 가능한 묶음



패키지

- ✓패키지 만들기
  - ✓패키지 폴더, 모듈, __init__.py 로 구성
    - ✓__init__.py
      - ✓__all__ = [' 모듈명 ' ,…]

패키지

✓[모듈 및 패키지 2]

✓1. [모듈 및 패키지 1] 문제의 gcdlcm 모듈과 [함수 5] 문제의 2부터 인수까지의 숫자 중 소수를 출력하는 함수 prime_print 가 있는 prime 모듈로 구성된 testpack 패키지를 만들어보자.

✓2. 해당 패키지를 활용하여 GcdLcm 객체를 생성한 뒤 gcd, lcm 함수를 사용해보고, prime_print 함수에 인수 100을 넣고 사용해보자.

예외 처리

한국외국어대학교 Social Science & AI융합학부

이 동 현

목차

✓예외 처리

예외 처리

- ✓예외 처리
  - ✓오류 처리를 위해 사용됨
  - ✓오류가 일어날 수 있는 코드 (try)와 오류 발생 시 실행되는 코드 부분 (except) 으로 크게 나눌 수 있음
    - ✓try, except, (else) 문
    - ✓try, except, finally 문
    - ✓raise, with 문 등이 있음

예외 처리

- ✓try, except, else 문
  - ✓try:

    # 오류가 일어날 수 있는 코드
  - ✓except [오류 타입 as 객체]:

    # 오류 발생시 실행 코드

    # 오류 타입 입력 시 해당 오류 타입 발생시 실행

    # as e 입력 시 오류 메시지를 객체 e에 받아 옴
  - ✓else:

    # 오류 없을 시 실행 코드

    # else 없이도 실행 가능

예외 처리

- ✓try, except, finally 문
  - ✓try:

    # 오류가 일어날 수 있는 코드
  - ✓except [오류 타입 as 객체]:

    # 오류 발생시 실행 코드

    # 오류 타입 입력 시 해당 오류 타입 발생시 실행
  - ✓finally:

    # 오류 발생 여부와 관계없이 실행

감사합니다.